Energy Efficiency of LLM-Generated Code: A Cross-Language and Cross-Platform Empirical Study

Brisild Capani
[TBD] Affiliation
brisild.capani@aeb.com

Seyyid [TBD]
[TBD] Affiliation
[TBD]

***Abstract*—**

TODO: Translate and condense the existing German abstract (~300 words) to ~200 words. Final numbers and findings can only be added after the experiments; until then, the abstract describes goal and method only.

Large Language Models (LLMs) are increasingly used to generate source code, yet their evaluation focuses predominantly on functional correctness. The energy consumed when the generated code is *executed* has received comparatively little attention. This paper presents an empirical study of the energy efficiency of LLM-generated code along three factors: the generating LLM (Claude, GPT, and Gemini, accessed via their official coding CLIs), the target programming language (Python, Java, C++), and the hardware platform on which the code runs (laptop, Raspberry Pi, server). Using a full factorial design over [TBD] programming tasks, we measure energy consumption, execution time, and peak memory usage of functionally correct solutions and analyse main and interaction effects with mixed-effects models. In particular, we investigate whether the relative ranking of LLMs with respect to energy efficiency is stable across languages and platforms.

TODO: Results and implications.

***Index Terms*—Large language models, code generation, energy efficiency, green software engineering, empirical study, programming languages, Raspberry Pi**

I. Introduction

LLM-based assistants have become part of everyday software development, generating anything from small snippets to complete functions. Benchmarks for code generation predominantly measure *functional correctness*, e.g., the share of generated solutions that pass a set of unit tests. Non-functional properties such as runtime performance and, in particular, energy efficiency are rarely considered, even though generated code may be executed many times in production and on a wide range of devices, from data-centre servers to resource-constrained embedded systems.

TODO: Add 1–2 citations on the ICT/software share of energy consumption and on the adoption of AI coding assistants.

Early studies indicate that the energy efficiency of LLM-generated code varies between models and often falls short of human-written reference solutions [1]–[3]. However, existing studies typically vary only a subset of the factors that plausibly influence energy consumption: they either cover several languages on a small number of similar platforms [1], or several heterogeneous platforms for a single language [3]. It therefore remains unclear whether a statement such as "model A generates more energy-efficient code than model B" holds in general, or only for a particular language and execution environment.

This paper addresses this question with a full factorial experiment in which three LLMs, accessed through their vendors' official coding CLIs, generate solutions for 14 programming tasks in Python, Java, and C++. All functionally correct solutions are executed on three heterogeneous platforms (an x86 laptop, an x86 server, and an ARM-based Raspberry Pi), and their energy consumption, execution time, and memory usage are measured under controlled conditions. Our central research question is:

> *To what extent do the choice of LLM, programming language, and hardware platform influence the energy efficiency of LLM-generated code?*

The contributions of this paper are:

- A full factorial empirical study of LLM-generated code across three LLMs (Claude, GPT, Gemini via their official CLIs), three programming languages, and three hardware platforms.
- A statistical analysis of main *and interaction* effects, with a particular focus on whether the energy-efficiency ranking of LLMs is stable across languages and platforms.
- An analysis of the relationship between energy, execution time, and memory for generated code.
- A replication package containing tasks, prompts, generated code, measurement scripts, and raw data.

TODO: Paper outline paragraph ("The remainder of this paper is structured as follows…") once the structure is final.

II. Background

A. LLM-Based Code Generation

TODO: Short overview: how LLMs generate code, common benchmarks (HumanEval, MBPP, EvalPlus, EvoEval, LeetCode-based sets), pass@k as correctness metric. Cite originals.

B. Software Energy Measurement

TODO: Energy vs. power ($E = \int P\,dt$); software-based estimation (Intel/AMD RAPL, tools building on it such as Scaphandre, CodeCarbon, perf) vs. hardware-based measurement (external power meters, INA219/INA226 shunt sensors). Explain which components each method captures (package, DRAM, whole system) and why RAPL is not available on ARM-based Raspberry Pi boards.

C. Energy Efficiency of Programming Languages

Pereira et al. [4] compared 27 programming languages on the Computer Language Benchmarks Game and examined how energy, time, and memory relate.

TODO: Summarise key methodological points relevant to our study (compiled vs. virtual-machine vs. interpreted languages; faster is not always more energy-efficient) – in own words, without anticipating our results.

III. Related Work

A. Energy Efficiency of LLM-Generated Code

a) Single-language studies.: Most empirical work on the energy efficiency of LLM-generated code targets Python on a single platform. Islam et al. [2] compare code generated by 20 LLMs for 878 LeetCode problems with canonical human solutions on an Intel Xeon server, using RAPL via `perf`, and find LLM solutions to be less energy-efficient on average, with large differences between models and algorithmic categories. Cappendijk et al. [5] and Ashraf et al. [6] study the effect of prompt modifications and of model size, respectively; both report that outcomes depend on the specific combination of model and problem. Vartziotis et al. [7] propose a "green capacity" metric for comparing code assistants.

b) Hardware platforms.: Cursaru et al. [8] measure code generated by a single model (Code Llama) for C++, JavaScript, and Python on a Raspberry Pi with an external power monitor and observe that efficiency depends strongly on language and problem. Apsan et al. [3] are, to our knowledge, the only study to execute LLM-generated code on three heterogeneous platforms (an Intel Xeon server, an Intel Core i9 PC, and an ARM Cortex-A72 Raspberry Pi), comparing six LLMs and four prompting techniques against human and expert-written Python code. Each platform is analysed separately, using aligned rank transform (ART) ANOVA with LLM and solution type as factors. The authors observe that the most efficient LLM differs between platforms, but this observation is not tested statistically, and the study is restricted to Python and nine problems.

c) Multiple languages.: Solovyeva et al. [1] evaluate three LLMs (GitHub Copilot, GPT-4o, o1-mini) on 53 hard LeetCode problems in Python, Java, and C++ on an Apple M3 laptop and an Intel PC, measuring CPU energy with `powermetrics` and RAPL. They use pairwise Mann–Whitney U tests and Spearman correlations between platforms; C++ programs are compiled without optimisation (`-O0`). Interactions between model, language, and platform are not modelled. Rekhi et al. [9] evaluate eight prompting strategies across ten LLMs for Python and C++ on a single Intel cloud platform and note qualitatively that the effect of prompting is inconsistent across models and languages, without inferential statistics.

d) Secondary studies.: A recent systematic literature review of 18 primary studies [10] concludes that evidence is "limited and highly heterogeneous", that most studies evaluate generated code on a single hardware configuration, that few use direct hardware-based measurement, and explicitly calls for cross-platform evaluations.

B. Runtime Efficiency Benchmarks

A parallel line of work proposes benchmarks for the runtime efficiency (execution time and memory, not energy) of generated code, e.g., EffiBench [11], EvalPerf [12], ENAMEL [13], and COFFE [14]. EffiBench-X [15] extends this to six languages, including Python, Java, and C++, and reports language-specific differences in the relative efficiency of LLMs. These benchmarks are candidate task sources for our study (Section V), but they measure on a single platform and do not report energy.

C. Energy Efficiency of Programming Languages

Pereira et al. [4], [16] rank up to 27 languages by energy, time, and memory on the Computer Language Benchmarks Game and show that the fastest language is not always the most energy-efficient. Their results concern human-written, highly optimised programs on a single x86 machine; whether similar relations hold for LLM-generated code and across architectures is open.

D. Research Gap

Table I summarises the closest studies. Three observations motivate our work:

1) *No study crosses multiple languages with architecturally heterogeneous platforms.* Multi-language studies use one or two x86/Apple platforms [1], [9]; the only three-platform study is Python-only [3].
2) *Interaction effects are not tested.* Several studies report that results "depend on" model, language, or platform [3], [5], [8], [9], but none models these dependencies as interaction terms or tests the stability of LLM rankings.
3) *Measurement and statistics are heterogeneous* [10]: several studies report averages without inferential statistics, and C++ is sometimes compiled without optimisation.

Our study addresses these points with a full factorial design (LLM $\times$ language $\times$ platform), a mixed-effects analysis with explicit interaction terms, and a test of rank stability across languages and platforms.

TODO: Verify Cappendijk et al. platform/tool and Cursaru et al. statistics from full texts. Re-run the arXiv/Scopus search shortly before submission.

IV. Research Questions and Hypotheses

RQ1 Which LLMs generate the most energy-efficient code for identical programming tasks?

RQ2 How does the energy efficiency of generated code differ between Python, Java, and C++?

RQ3 How does the relative energy efficiency and execution time of generated code differ across a laptop, a server, and a Raspberry Pi?

RQ4 Are there statistically significant interaction effects between LLM, programming language, and hardware platform, i.e., does the ranking of LLMs change depending on language and platform?

RQ5 How are execution time, energy consumption, and memory usage related?

TABLE I
CLOSEST EMPIRICAL STUDIES ON THE ENERGY EFFICIENCY OF LLM-GENERATED CODE. EXT. = EXTERNAL POWER METER; RAPL = SOFTWARE-BASED (INTEL RAPL).

Study	LLMs	Languages	Platforms	Measurement	Interact.
Cursaru et al. [8]	1	C++, JS, Py	RPi	Ext.	[TBD]
Cappendijk et al. [5]	5	Py	1 (x86)	RAPL	no
Islam et al. [2]	20	Py	1 (x86 server)	RAPL	no
Solovyeva et al. [1]	3	Py, Java, C++	Mac (M3), PC (x86)	powermetrics, RAPL	no
Apsan et al. [3]	6	Py	Server, PC, RPi	EnergiBridge, Ext.	partly[a]
Rekhi et al. [9]	10	Py, C++	1 (x86 cloud)	RAPL	no
This study	3[b]	Py, Java, C++	Laptop, Server, RPi	RAPL, PMIC	yes

[a] LLM × prompt interaction tested within each platform; no interaction with platform. [b] Accessed via the vendors' official coding CLIs.

Each research question is accompanied by a hypothesis (null hypotheses $H_{x,0}$ state the absence of the effect). All hypotheses are tested; none is presupposed to hold.

H1 (RQ1) Energy consumption of generated code differs between LLMs (main effect of LLM).

H2 (RQ2, RQ4) The effect of the LLM on energy consumption depends on the programming language (LLM × language interaction).

H3 (RQ3, RQ4) The effect of the LLM on energy consumption depends on the hardware platform (LLM × platform interaction).

H4 (RQ4) There is a three-way interaction LLM × language × platform.

H5 (RQ5) Execution time and energy consumption are correlated, and the strength of this relation differs between languages and platforms.

TODO: Decide whether language and platform main effects (RQ2/RQ3) get their own hypotheses. Note: the language main effect is largely expected from prior work [4]; the novelty lies in the interactions.

TODO: Pre-register hypotheses and analysis plan (e.g., OSF) before data collection.

V. EXPERIMENTAL SETUP

A. Study Design

We use a full factorial design with three fixed factors: *LLM* (three levels), *programming language* (Python, Java, C++), and *hardware platform* (laptop, server, Raspberry Pi 5). Programming tasks (14) are treated as a random factor, and each task–LLM–language combination is sampled $k = 5$ times, yielding $3 \times 3 \times 14 \times 5 = 630$ generated programs. Every functionally correct program is executed on all three platforms. The dependent variables are energy consumption (primary), execution time, peak memory usage, and functional correctness.

B. LLMs and Access via Official Coding CLIs

We study the flagship models of the three most widely used commercial AI assistants: Claude (Anthropic), ChatGPT (OpenAI), and Gemini (Google). Rather than querying the bare models through their APIs, we access each model through its vendor's official command-line coding agent (Claude Code, Codex CLI, Gemini CLI), authenticated with a regular subscription (Table II). This reflects how developers increasingly obtain generated code in practice; the factor *LLM* therefore denotes the combination of model and official tool. Open-weight models, which lack a comparable official subscription-based coding CLI, are left to future work, so that all levels of the factor share the same mode of access.

TABLE II
LLMS AND THE OFFICIAL CLIS USED TO ACCESS THEM. EXACT MODEL IDENTIFIERS AND CLI VERSIONS WERE RECORDED AT GENERATION TIME.

Vendor	CLI (version)	Model	Identifier
Anthropic	Claude Code ([TBD])	Claude Opus 5.5	`claude-opus-5-5`
OpenAI	Codex CLI ([TBD])	GPT flagship[a]	[TBD]
Google	Gemini CLI ([TBD])	Gemini Pro[a]	[TBD]

[a] Most capable model offered by the CLI for the subscription at generation time.

To isolate the models' code generation from agentic behaviour, every request is issued non-interactively in a fresh, empty working directory, without user configuration, project context files, or session history, and with tool use disabled where the CLI supports it (Claude Code, Gemini CLI) or restricted to a read-only sandbox otherwise (Codex CLI). The CLIs' built-in system prompts are left unchanged. We log every request's full event stream and report whether any tool was invoked. Details are given in Section VI-A.

C. Programming Tasks

The task set (Table III) combines two complementary sources.

a) EffiBench-X tasks (algorithm free).: Ten tasks are drawn from EffiBench-X [15], which provides problem statements, test-case generators including stress inputs, and canonical solutions in Python, Java, and C++. We restrict the pool to the 244 function-style problems (originally from LeetCode) of difficulty *Medium* or *Hard* for which starter code, a canonical solution, and a test runner exist in all three languages. The selection is stratified by category and difficulty: for each of five categories, defined by the problems' LeetCode tags, one Medium and one Hard problem is drawn uniformly at

random (seed `20261025`); a problem drawn for one category is removed from the pool for the following categories. Since problems often carry several tags, the categories describe the selection strata rather than an exclusive classification.

Function-style tasks let all LLMs share the same, language-specific input/output harness supplied by EffiBench-X, so differences in measured energy stem from the generated solution rather than from differing I/O code. The LLM is free to choose any algorithm.

b) CLBG tasks (algorithm prescribed).: Four tasks are taken from the Computer Language Benchmarks Game (CLBG): `n-body`, `spectral-norm`, `binary-trees`, and `fannkuch-redux`. They are CPU- and memory-intensive, scale to arbitrary input sizes, and link our study to prior work on language energy efficiency [4], [16]. In contrast to the EffiBench-X tasks, their specifications prescribe the algorithm, so differences between LLMs reflect implementation choices only. We rewrote the specifications as self-contained task descriptions and verified them with our own reference implementations against the official CLBG reference outputs.

c) Workload.: For each task we fix a single input that is identical across LLMs, languages, and platforms: for EffiBench-X tasks a large input produced by the task's test generator, for CLBG tasks the parameter N. Its size is calibrated in a pilot study such that the C++ reference solution runs for at least [TBD] s on the server, keeping measurements well above the resolution of the energy counters.

D. Hardware Platforms and Software Environment

Table IV summarises the platforms. To avoid confounding platform with runtime versions, all programs are built and executed in identical, version-pinned multi-architecture container images (amd64/arm64) on all platforms. C++ is compiled with `-O2` and without sanitizers.

E. Energy Measurement

On the x86 platforms, energy is read from the Running Average Power Limit (RAPL) interface via `perf stat`, capturing the CPU package domain and, where supported, the DRAM domain [17]. The Raspberry Pi 5 offers no RAPL; instead, we sample the voltage and current channels of its power-management IC at approximately [TBD] Hz and integrate their summed power over the run.

TODO: If a USB-C power meter is acquired: describe calibration of PMIC readings against it.

Because these methods capture different system boundaries, *absolute* energy values are not compared across platforms. Cross-platform questions (**RQ3**, **RQ4**) are answered using relative measures: LLM rankings, ratios to the reference solution, and energy normalised within each platform.

a) Controlled conditions.: On all platforms, the CPU frequency governor is set to `performance`, turbo/boost is disabled where possible, the measured process is pinned to dedicated cores, and non-essential services are stopped. Before each measurement block, idle power is recorded for 60 s; we report both gross and idle-corrected energy. Execution order is randomised across LLM, language, and task to prevent drift from being confounded with the factors, with a cool-down of [TBD] s between runs; CPU temperature is logged.

VI. Methodology and Data Collection

A. Prompting and Code Generation

All models receive the same zero-shot prompt, instantiated from one of two templates (Figure 1 in the appendix): a *function* template for EffiBench-X tasks, which supplies the problem statement and the language-specific code skeleton, and a *program* template for CLBG tasks, which asks for a complete program taking N as a command-line argument. Within a template, prompts for different languages differ only in the language name, one line stating the language version and compiler command, and the code skeleton. Both templates require standard-library-only, single-threaded code and deliberately contain no reference to efficiency, speed, or energy, as we study the models' default behaviour. The prompt is passed unchanged to each CLI as the single user message.

EffiBench-X problem statements are converted from the original HTML to Markdown; we use the HTML rather than the dataset's Markdown field, in which superscripts are lost (e.g., the constraint 10^5 appears as "105"), which could mislead models in their choice of algorithm. Two tasks (EB05, EB07) contain illustrative images, which are replaced by the placeholder "[image omitted]"; their examples remain fully specified in text. All 42 rendered prompts and their SHA-256 hashes are part of the replication package.

Each request is issued as a separate, non-interactive CLI invocation (Claude Code: `claude -p`; Codex CLI: `codex exec`; Gemini CLI: `gemini -p`) with the following controls: a fresh, empty working directory; user and project configuration ignored; session persistence disabled; the model fixed explicitly; tool use disabled (`-tools ""` in Claude Code; tool exclusion in the Gemini CLI settings) or, where no such option exists, a read-only sandbox (Codex CLI); and structured JSON output. Sampling parameters and reasoning effort are left at each tool's defaults, which cannot be set uniformly across tools. The CLI versions are pinned for the duration of data collection, and the model identifier reported in each response is recorded.

TODO: Verify exact Codex/Gemini options after installation and list the final command lines in the replication package.

For each task, model, and language we draw $k = 5$ independent samples, each in a new session. The source code is extracted automatically from the first fenced code block of the final response; no manual or automated repair is applied. All prompts, raw event streams, timestamps, and token counts (where reported) are logged.

B. Functional Correctness

Each program is compiled (C++, Java) and executed against the task's full test suite (EffiBench-X: generated tests; CLBG: reference output) inside the same container images used for measurement. A program is *correct* if it compiles and passes all tests within the time limit. Correctness rates are reported per LLM and language. Only correct programs enter the energy

TABLE III
PROGRAMMING TASKS. EB = EFFIBENCH-X (FUNCTION, ALGORITHM FREE; RELEASE DATE OF THE ORIGINAL LEETCODE PROBLEM); CLBG = COMPUTER LANGUAGE BENCHMARKS GAME (PROGRAM, ALGORITHM PRESCRIBED).

ID	Task	Stratum / category	Difficulty	Released
EB01	Maximum Total Damage With Spell Casting	Sorting / searching	Medium	2024-06
EB02	Kth Smallest Amount With Single Denomination Combination	Sorting / searching	Hard	2024-04
EB03	Sum of Digit Differences of All Pairs	Mathematics	Medium	2024-05
EB04	Count the Number of Infection Sequences	Mathematics	Hard	2023-12
EB05	Find Subtree Sizes After Changes	String processing	Medium	2024-10
EB06	Palindrome Rearrangement Queries	String processing	Hard	2023-12
EB07	Maximum Score After Applying Operations on a Tree	Dynamic programming / recursion	Medium	2023-11
EB08	Count the Number of Powerful Integers	Dynamic programming / recursion	Hard	2024-01
EB09	Minimum Number of Coins for Fruits	Data structures	Medium	2023-11
EB10	Longest Common Suffix Queries	Data structures	Hard	2024-03
CLBG01	n-body	Floating-point simulation	–	–
CLBG02	spectral-norm	Floating-point linear algebra	–	–
CLBG03	binary-trees	Memory allocation / recursion	–	–
CLBG04	fannkuch-redux	Permutations / integer arithmetic	–	–

TABLE IV
HARDWARE AND SOFTWARE CONFIGURATION.

	Laptop	Server	Raspberry Pi 5
CPU / ISA	Intel Core i7-1185G7 (x86-64)	[TBD] (x86-64)	Cortex-A76 (Arm64)
RAM	32 GB	[TBD]	8 GB
OS / kernel	[TBD]	[TBD]	[TBD]
Python	CPython 3.14 (`python:3.14-slim`) [TBD]		
Java	OpenJDK 25 (`eclipse-temurin:25-jdk`) [TBD]		
C++	GCC 14, `-O2 -std=c++20` [TBD]		
Energy measurement	RAPL (`perf`)	RAPL (`perf`)	PMIC ADCs (`vcgencmd`)
Measured scope	CPU package (+DRAM)	CPU package + DRAM	board input

analysis. Rank-based analyses (**RQ4**) are restricted to tasks for which every LLM produced at least one correct solution in every language; the mixed-effects analysis uses all correct programs.

C. Measurement Procedure

Each correct program is executed 10 times per platform on the fixed task input. We measure the complete process, including interpreter or JVM start-up, since this reflects how the program would be executed in practice; start-up cost of a minimal program per language is reported separately. Per run we record energy, wall-clock time, peak resident memory, and CPU time. Runs exceeding a timeout of [TBD] min are recorded as timeouts. A pilot study (two LLMs, three tasks, all languages) is used to calibrate input sizes and to confirm the number of repetitions based on the observed coefficient of variation.

D. Statistical Analysis

TODO: Finalise and pre-register before data collection.

1) *Descriptive statistics*: median and interquartile range per cell.
2) *Main and interaction effects* (**RQ1**–**RQ4**): per platform, and jointly on platform-normalised energy, a linear mixed-effects model on log-transformed energy with fixed effects LLM, language, platform and all interactions, and random intercepts for task and for generated program (nested in task). Interactions are tested with likelihood-ratio tests; effect sizes and post-hoc contrasts are reported with Holm correction. If model assumptions are violated, aligned rank transform ANOVA serves as a non-parametric alternative.
3) *Rank stability* (**RQ4**): LLM rankings by median energy are computed per language–platform cell and compared using Kendall's W (overall concordance) and pairwise Kendall's τ.
4) *Energy, time, and memory* (**RQ5**): Spearman correlations per language and platform.

VII. RESULTS

A. Functional Correctness

TODO: Correctness rates per LLM × language.

B. RQ1: Differences between LLMs

TODO:

C. RQ2: Differences between Programming Languages

TODO:

D. RQ3: Differences between Hardware Platforms

TODO:

E. RQ4: Interaction Effects and Rank Stability

TODO:

F. RQ5: Relation of Energy, Time, and Memory

TODO:

VIII. Discussion

TODO: Interpret results with respect to the RQs; relate to prior work [1]–[4]; analyse *why* differences arise (e.g., algorithmic choices, library usage, data structures) by qualitatively inspecting selected solutions; implications for developers and for LLM benchmarking.

IX. Threats to Validity

a) Construct validity.: Energy measurement methods differ in which components they capture (CPU package and DRAM via RAPL vs. whole-board power via external meters), which limits the comparability of absolute values across platforms.

TODO: Describe mitigation chosen in Section V.

b) Internal validity.: Measurements may be influenced by background processes, thermal state, frequency scaling, JIT warm-up, and garbage collection. LLM outputs are non-deterministic.

TODO: Mitigations: repetitions, randomised order, fixed governor, multiple samples per task.

c) External validity.: Results are limited to the selected LLMs (and their versions at the time of the study), tasks, three languages, and three platforms, and may not generalise to other models, languages, or architectures, or to larger, real-world codebases.

d) Conclusion validity.:

TODO: Multiple testing, assumption violations, sample size / power.

e) Access via coding CLIs.: Models are accessed through their vendors' coding CLIs, whose built-in system prompts we cannot inspect or align, and whose defaults (e.g., sampling, reasoning effort) differ. The factor *LLM* thus measures model and tool jointly. We mitigate agentic effects by disabling tools, ignoring user and project configuration, and logging every tool invocation; Codex CLI offers no documented switch to disable tools, so it runs in a read-only sandbox instead. CLIs and models evolve quickly, so we pin CLI versions and record model identifiers and dates. With three levels of the LLM factor, rank-based analyses (**RQ4**) have limited resolution; the interaction terms of the mixed-effects model are therefore the primary evidence.

f) Confounding of model and implementation.: The effect of an LLM cannot be fully separated from the concrete algorithm it chooses; these strategy differences are, however, part of the phenomenon under study.

g) Data contamination.: The EffiBench-X tasks were released between November 2023 and October 2024 and the CLBG tasks and their solutions have been public for many years; all studied LLMs were likely trained on them. Memorised solutions may therefore inflate correctness and affect efficiency. Since all LLMs are exposed to the same tasks, this threat primarily concerns absolute results rather than the comparison between models; the contrast between algorithm-free (EffiBench-X) and algorithm-prescribed (CLBG) tasks further helps to interpret it.

TODO: Optionally add 2–3 problems released after all model cutoffs as a contamination check.

X. Conclusion and Future Work

TODO: Summarise answers to RQ1–RQ5 once results are available.

a) Future work.: Open-weight models (e.g., Kimi, DeepSeek, Qwen), accessed under comparable conditions, and a comparison of bare API access with CLI access to the same model are natural extensions.

TODO: Further: energy-aware prompting, additional languages (Rust, Go, JavaScript), larger real-world tasks, energy cost of generation vs. execution.

b) Data availability.:

TODO: Link to replication package (tasks, prompts, generated code, scripts, raw data).

References

[1] L. Solovyeva, S. Weidmann, and F. Castor, "AI-powered, but power-hungry? Energy efficiency of LLM-generated code," arXiv preprint arXiv:2502.02412, 2025. [Online]. Available: https://arxiv.org/abs/2502.02412

[2] M. A. Islam, D. V. Jonnala, R. Rekhi, P. Pokharel, S. Cilamkoti, A. Imran, T. Kosar, and B. Turkkan, "Evaluating the energy-efficiency of the code generated by LLMs," arXiv preprint arXiv:2505.20324, 2025. [Online]. Available: https://arxiv.org/abs/2505.20324

[3] R. Apsan, V. Stoico, M. Albonico, R. Dhar, K. Vaidhyanathan, and I. Malavolta, "Generating energy-efficient code via large-language models – where are we now?" in *Proceedings of the IEEE/ACM 48th International Conference on Software Engineering (ICSE)*. ACM, 2026, pp. 426–439, preprint: arXiv:2509.10099.

[4] R. Pereira, M. Couto, F. Ribeiro, R. Rua, J. Cunha, J. P. Fernandes, and J. Saraiva, "Energy efficiency across programming languages: How do energy, time, and memory relate?" in *Proceedings of the 10th ACM SIGPLAN International Conference on Software Language Engineering (SLE)*. New York, NY, USA: ACM, 2017, pp. 256–267.

[5] T. Cappendijk, P. de Reus, and A. Oprescu, "An exploration of prompting LLMs to generate energy-efficient code," in *Proceedings of the IEEE/ACM 9th International Workshop on Green and Sustainable Software (GREENS)*. IEEE, 2025, pp. 31–38.

[6] H. Ashraf, S. M. Danish, A. Leivadeas, Y. Otoum, and Z. Sattar, "Energy-aware code generation with LLMs: Benchmarking small vs. large language models for sustainable AI programming," arXiv preprint arXiv:2508.08332, 2025. [Online]. Available: https://arxiv.org/abs/2508.08332

[7] T. Vartziotis, I. Dellatolas, G. Dasoulas, M. Schmidt, F. Schneider, T. Hoffmann, S. Kotsopoulos, and M. Keckeisen, "Learn to code sustainably: An empirical study on LLM-based green code generation," arXiv preprint arXiv:2403.03344, 2024. [Online]. Available: https://arxiv.org/abs/2403.03344

[8] V.-A. Cursaru, L. Duits, J. Milligan, D. Ural, B. Rodriguez Sanchez, V. Stoico, and I. Malavolta, "A controlled experiment on the energy efficiency of the source code generated by Code Llama," arXiv preprint arXiv:2405.03616, 2024. [Online]. Available: https://arxiv.org/abs/2405.03616

[9] R. Rekhi, B. Zhang, M. A. Islam, J. K. Pasham, Y. Chitturi, A. Paramesha, I. Valiveti, A. Imran, B. Turkkan, and T. Kosar, "Improving the energy-efficiency of the code generated by LLMs through effective prompting," arXiv preprint arXiv:2610.02571, 2026. [Online]. Available: https://arxiv.org/abs/2610.02571

[10] S. B. Masthan Ali, S. M. Danish, A. Hameed, O. Kirmani, and G. Srivastava, "Energy-efficient code generation using large language models: A systematic literature review," arXiv preprint arXiv:2603.00989, 2026. [Online]. Available: https://arxiv.org/abs/2603.00989

[11] D. Huang, Y. Qing, W. Shang, H. Cui, and J. M. Zhang, "EffiBench: Benchmarking the efficiency of automatically generated code," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2024, arXiv:2402.02037.

[12] J. Liu, S. Xie, J. Wang, Y. Wei, Y. Ding, and L. Zhang, "Evaluating language models for efficient code generation," arXiv preprint arXiv:2408.06450, 2024. [Online]. Available: https://arxiv.org/abs/2408.06450

[13] R. Qiu, W. W. Zeng, J. Ezick, C. Lott, and H. Tong, "How efficient is LLM-generated code? A rigorous & high-standard benchmark," in *International Conference on Learning Representations (ICLR)*, 2025, arXiv:2406.06647.
[14] Y. Peng, J. Wan, Y. Li, and X. Ren, "COFFE: A code efficiency benchmark for code generation," arXiv preprint arXiv:2502.02827. Accepted at FSE 2025, 2025. [Online]. Available: https://arxiv.org/abs/2502.02827
[15] Y. Qing, B. Zhu, M. Du, Z. Guo, T. Y. Zhuo, Q. Zhang, J. M. Zhang, H. Cui, S.-M. Yiu, D. Huang, S.-K. Ng, and L. A. Tuan, "EffiBench-X: A multi-language benchmark for measuring efficiency of LLM-generated code," arXiv preprint arXiv:2505.13004, 2025. [Online]. Available: https://arxiv.org/abs/2505.13004
[16] R. Pereira, M. Couto, F. Ribeiro, R. Rua, J. Cunha, J. P. Fernandes, and J. Saraiva, "Ranking programming languages by energy efficiency," *Science of Computer Programming*, vol. 205, p. 102609, 2021.
[17] G. Raffin and D. Trystram, "Dissecting the software-based measurement of CPU energy consumption: A comparative analysis," arXiv preprint arXiv:2401.15985, 2024. [Online]. Available: https://arxiv.org/abs/2401.15985

APPENDIX

Figure 1 shows both prompt templates verbatim; they are included directly from the experiment repository (`experiment/prompts/`). Placeholders in braces are filled per task and language: `{language}` is *Python*, *Java*, or *C++*; `{language_note}` is given in Table V; `{problem}` is the problem statement (EffiBench-X) or the task specification (CLBG); `{starter}` is the EffiBench-X code skeleton for the respective language. Figure 2 shows a fully rendered prompt.

TABLE V
LANGUAGE-SPECIFIC LINE `{LANGUAGE_NOTE}`.

Language	Text
Python	Target Python 3.14 (CPython).
Java	Target Java 25 (OpenJDK). *Program template only:* The public class must be named `Main`.
C++	Target C++20. The code is compiled with `g++ -O2 -std=c++20`.

(a) Function template (EffiBench-X tasks)

```
Solve the following programming problem in {language}.

Complete the code skeleton given below. Your code will be combined with a test harness
that reads the input, calls your method, and prints the result. Therefore, do not read
from standard input, do not write to standard output, and do not add a main function.

Requirements:
- Return the complete `Solution` class, including any import or include statements it needs.
- Use only the {language} standard library.
- Do not use multiple threads or processes.
- {language_note}
- Respond with exactly one fenced code block containing the code, and nothing else.

Problem:

{problem}

Code skeleton:

```{fence}
{starter}
```
```

(b) Program template (CLBG tasks)

```
Write a complete {language} program that solves the following task.

Requirements:
- The program reads a single integer N from its first command-line argument.
- It writes its result to standard output, exactly in the format specified below.
- Use only the {language} standard library.
- Do not use multiple threads or processes.
- {language_note}
- Respond with exactly one fenced code block containing the code, and nothing else.

Task:

{problem}
```

Fig. 1. Prompt templates used for all LLMs.

````
Solve the following programming problem in C++.

Complete the code skeleton given below. Your code will be combined with a test harness
that reads the input, calls your method, and prints the result. Therefore, do not read
from standard input, do not write to standard output, and do not add a main function.

Requirements:
- Return the complete `Solution` class, including any import or include statements it needs.
- Use only the C++ standard library.
- Do not use multiple threads or processes.
- Target C++20. The code is compiled with `g++ -O2 -std=c++20`.
- Respond with exactly one fenced code block containing the code, and nothing else.

Problem:

You are given an array `nums` consisting of **positive** integers where all integers have the **same** number of digits.

The **digit difference** between two integers is the *count* of different digits that are in the **same** position in
↪  the two integers.

Return the **sum** of the **digit differences** between **all** pairs of integers in `nums`.

**Example 1:**

**Input:** nums = [13,23,12]

**Output:** 4

**Explanation:**
We have the following:
- The digit difference between **1**3 and **2**3 is 1.
- The digit difference between 1**3** and 1**2** is 1.
- The digit difference between **23** and **12** is 2.
So the total sum of digit differences between all pairs of integers is `1 + 1 + 2 = 4`.

**Example 2:**

**Input:** nums = [10,10,10,10]

**Output:** 0

**Explanation:**
All the integers in the array are the same. So the total sum of digit differences between all pairs of integers will be
↪  0.

**Constraints:**

- `2 <= nums.length <= 10^5`
- `1 <= nums[i] < 10^9`
- All integers in `nums` have the same number of digits.

Code skeleton:

```cpp
class Solution {
public:
    long long sumDigitDifferences(vector<int>& nums) {

    }
};
```
````

Fig. 2. Fully rendered prompt for task EB03 (Sum of Digit Differences of All Pairs) in C++.

OPEN TODOS

- **TODO:** Translate and condense the existing German abstract (~300 words) to ~200 words. Final numbers and findings can only be added after the experiments; until then, the abstract describes goal and method only. 1
- **TODO:** Results and implications. 1
- **TODO:** Add 1–2 citations on the ICT/software share of energy consumption and on the adoption of AI coding assistants. 1
- **TODO:** Paper outline paragraph ("The remainder of this paper is structured as follows…") once the structure is final. 1
- **TODO:** Short overview: how LLMs generate code, common benchmarks (HumanEval, MBPP, EvalPlus, EvoEval, LeetCode-based sets), pass@k as correctness metric. Cite originals. 1
- **TODO:** Energy vs. power ($E = \int P\,dt$); software-based estimation (Intel/AMD RAPL, tools building on it such as Scaphandre, CodeCarbon, perf) vs. hardware-based measurement (external power meters, INA219/INA226 shunt sensors). Explain which components each method captures (package, DRAM, whole system) and why RAPL is not available on ARM-based Raspberry Pi boards. 1
- **TODO:** Summarise key methodological points relevant to our study (compiled vs. virtual-machine vs. interpreted languages; faster is not always more energy-efficient) – in own words, without anticipating our results. 2
- **TODO:** Verify Cappendijk et al. platform/tool and Cursaru et al. statistics from full texts. Re-run the arXiv/Scopus search shortly before submission. 2
- **TODO:** Decide whether language and platform main effects (RQ2/RQ3) get their own hypotheses. Note: the language main effect is largely expected from prior work [4]; the novelty lies in the interactions. 3
- **TODO:** Pre-register hypotheses and analysis plan (e.g., OSF) before data collection. 3
- **TODO:** If a USB-C power meter is acquired: describe calibration of PMIC readings against it. 4
- **TODO:** Verify exact Codex/Gemini options after installation and list the final command lines in the replication package. 4
- **TODO:** Finalise and pre-register before data collection. 5
- **TODO:** Correctness rates per LLM $\times$ language. 5
- **TODO:** 5
- **TODO:** 5
- **TODO:** 5
- **TODO:** 5
- **TODO:** 6
- **TODO:** Interpret results with respect to the RQs; relate to prior work [1]–[4]; analyse *why* differences arise (e.g., algorithmic choices, library usage, data structures) by qualitatively inspecting selected solutions; implications for developers and for LLM benchmarking. 6
- **TODO:** Describe mitigation chosen in Section V. 6
- **TODO:** Mitigations: repetitions, randomised order, fixed governor, multiple samples per task. 6
- **TODO:** Multiple testing, assumption violations, sample size / power. 6
- **TODO:** Optionally add 2–3 problems released after all model cutoffs as a contamination check. 6
- **TODO:** Summarise answers to RQ1–RQ5 once results are available. 6
- **TODO:** Further: energy-aware prompting, additional languages (Rust, Go, JavaScript), larger real-world tasks, energy cost of generation vs. execution. 6
- **TODO:** Link to replication package (tasks, prompts, generated code, scripts, raw data). 6